

Week 5 – Coordinate transforms for robotics

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



**WE ARE
HUMBER**

Review from Last Week



- **This week we covered Performance objectives**
 - Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
 - Configure ROS2 on a Linux-based system
 - Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
 - Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices
- **Next week's topics**
 - What is a ROS2 workspace?
 - What is a ROS2 package?
- **Next week's performance objectives**
 - Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
 - Configure ROS2 on a Linux-based system
 - Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
 - Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices
 - Utilize ROS2 topics, services and actions for inter-node communication

Agenda for today's training session



1. Why Coordinate Transformations?

- Representing robots mathematically
- From complex problems → simple equations

2. Representing Robots as Points

- Single point vs multiple points
- 2D vs 3D representations

3. Transforming Points with Functions

- 1D → 2D → 3D functions
- Mapping inputs to outputs

4. 2D & 3D Functions

- Extending dimensionality
- Translation & rotation as core transforms

5. Linear Transformations

- Identity, scaling, shearing
- Properties & chaining

6. 2D Rotations

- Derivation of rotation matrix
- Key properties (transpose, determinant, composition)

7. 3D Rotations

- R_x, R_y, R_z matrices
- Composition rules & SO(3)

8. Translations & Homogeneous Coordinates

- Translation problem
- Homogeneous solution (2D & 3D)

Review: ROS 2 Workspace and Package Architecture

- - A workspace is the top-level folder where ROS 2 packages are built.
- Typical folders after building:
 - **src/** → source code (packages)
 - **build/** → temporary build files
 - **install/** → final executables, setup files
 - **log/** → build and run logs

Commands:

```
mkdir -p test_ws/src
```

```
cd test_ws
```

```
colcon build
```

Review: Creating a C++ Package

- Inside `src/`, create a C++ package:
`ros2 pkg create --build-type ament_cmake test_cpp`

Auto-generated files:

- `package.xml` → package metadata
- `CMakeLists.txt` → build instructions
- `src/` & `include/` → where C++ code lives

Example structure:

`test_cpp/`

- `CMakeLists.txt`
- `package.xml`
- `include/test_cpp/`
- `src/`

Review: Creating a Python Package

- Inside `src/`, create a Python package:
`ros2 pkg create --build-type ament_python test_py`

Auto-generated files:

- `setup.py` → installation setup
- `package.xml` → metadata
- `resource/` & `test/` folders
- Python module with `__init__.py`

Example structure:

```
test_py/  
├── package.xml  
├── setup.py  
├── resource/test_py  
├── test/  
└── test_py/__init__.py
```

Review: Building the Workspace

- Go back to workspace root and build:

```
cd ~/test_ws
```

```
colcon build
```

Output:

- install/ gets executables and setup files
- Each package gets its own folder inside install/share

Review: Sourcing the Environment

- **After building, source the setup file:**
source install/setup.bash

This updates your shell so ROS 2 can find your packages.

Review: Running Your Node

- **Run executables/nodes inside your package:**
ros2 run test_cpp test_node

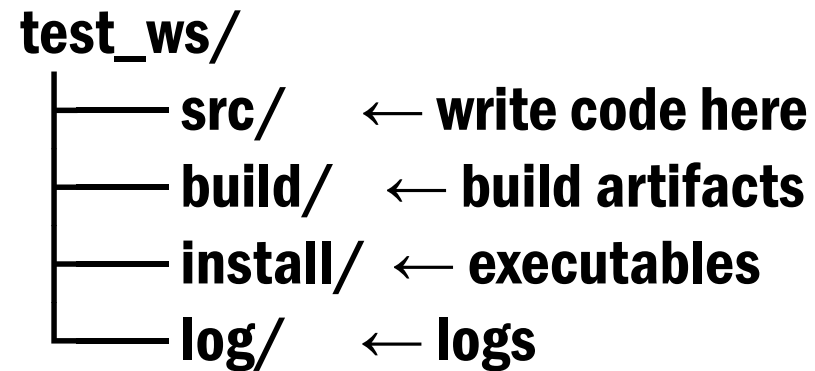
Example output:

hello world test_cpp package

Review: ROS 2 Folder Flow (Big Picture)

- **Flow of development:**
 - 1. Create a workspace (test_ws/)**
 - 2. Add packages inside src/**
 - C++ → ament_cmake
 - Python → ament_python
 - 3. Build with colcon build**
 - 4. Source install/setup.bash**
 - 5. Run nodes with ros2 run**

Diagram:



Coordinate transforms : Tools

- GeoGebra <https://www.geogebra.org/>

GeoGebra 3D Calculator

ThetaZ = -3

-3 3

$$\text{RotMatZ} = \begin{pmatrix} \cos(\text{ThetaZ}) & -\sin(\text{ThetaZ}) & 0 \\ \sin(\text{ThetaZ}) & \cos(\text{ThetaZ}) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} -0.99 & 0.14 & 0 \\ -0.14 & -0.99 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

A = (1, 1, -1)

A2 = RotMatY RotMatZ A

= (-0.85, -1.13, -1)

Input...

ASSIGN

Coordinate transforms : Introduction



- Robotics = representing the **physical world mathematically**
- Needed for:
 - ✓ Designing control systems
 - ✓ Planning trajectories
 - ✓ Predicting motion
- Multiple representations depending on the problem



Introduction



- Building a **mathematical “language”** to describe a robot's **shape & movement in space**
- Core tools: **positions + rotations**



Introduction



- Applications:
 - Simulation & visualization
 - Structure for processing sensor data
 - Actuator control
- Strong foundations simplify complex problems → few lines of math

Introduction



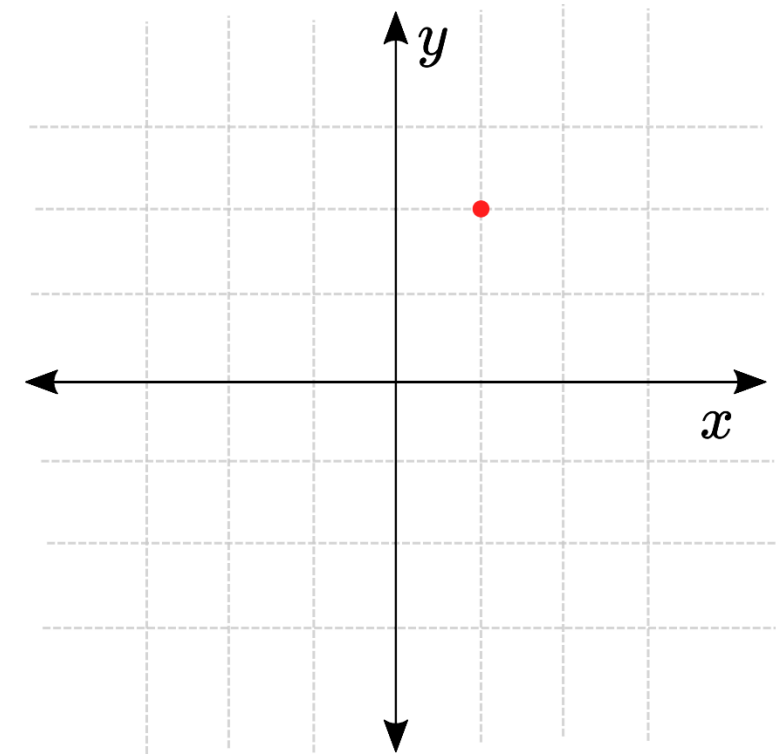
- Robots = points in space (coordinates)
- Study how to:
 - ✓ Move them $\rightarrow$ translations
 - ✓ Orient them $\rightarrow$ rotations
- Together = coordinate transformations



Representing a Robot in Space



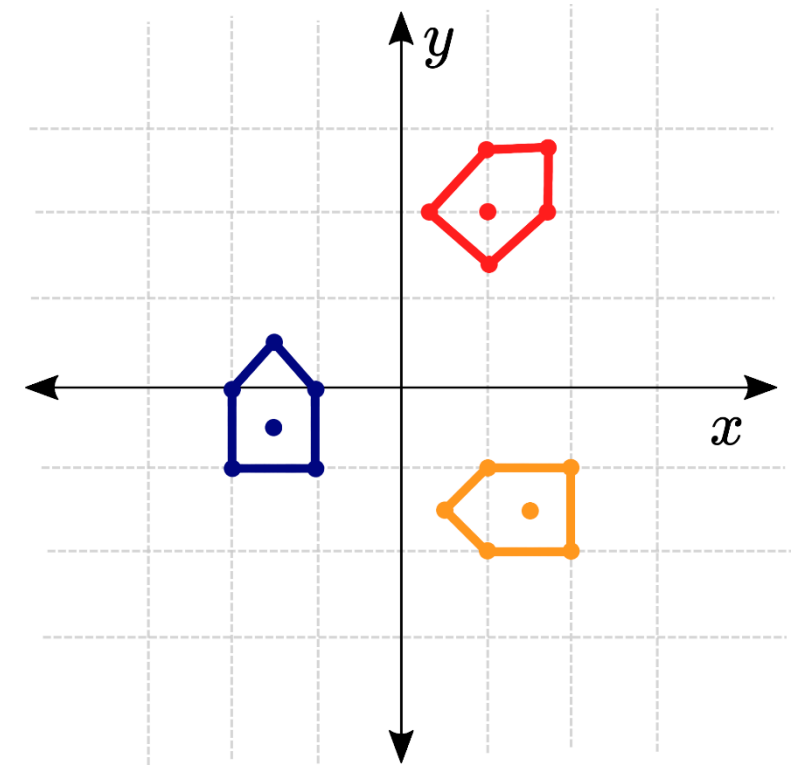
- Step 1: Decide how to describe the robot
- Simplest: a single point → shows position only
- Limitation: no rotation, no shape, no scale



Using Multiple Points



- Series of points = better representation
- Could be:
 - Simple shapes (rectangle, triangle)
 - Complex CAD models
- Captures size + orientation



2D vs 3D Representations

- Many robots: movement on a **plane** → 2D is enough
- Others (e.g., drones): need **3D**
- Always depends on system's motion constraints



[Source](#)

Introducing Coordinate Transforms

- Start: robots as points moving in space
- Next: multiple **coordinate systems & reference frames**
- Helps simplify math for complex systems

Mathematical Representation

- **Points as column vectors**

- **In 2D:**

$$p_{2D} = \begin{bmatrix} x \\ y \end{bmatrix}$$

- **In 3D:**

$$p_{3D} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- **Cartesian most common (though polar sometimes useful)**

Mathematical Representation

- **With points defined, we can now:**
 - **Manipulate them**
 - **Explore translations & rotations**
 - **Describe robot motion through transformations**

Transforming Points with a Function: Introduction

- A **function** maps inputs $\rightarrow$ outputs
- Common in maths & engineering
- Foundation for coordinate transformations

- General form:

$$y = f(x, t)$$

- Inputs: scalar(s) $\rightarrow$ Output: scalar

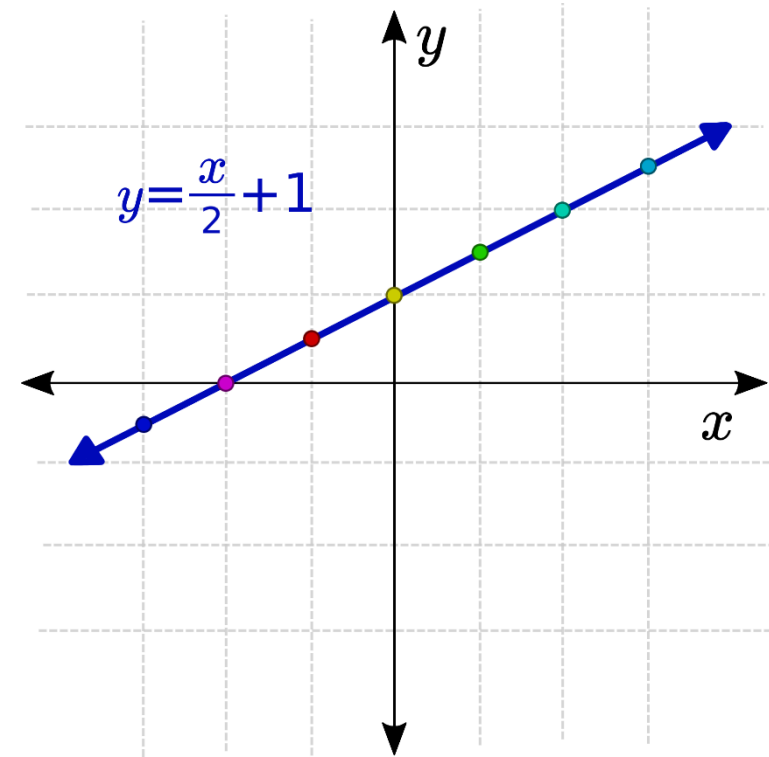
- Examples:

- $y = x^2 + 1$

- $y = \sin(x)$

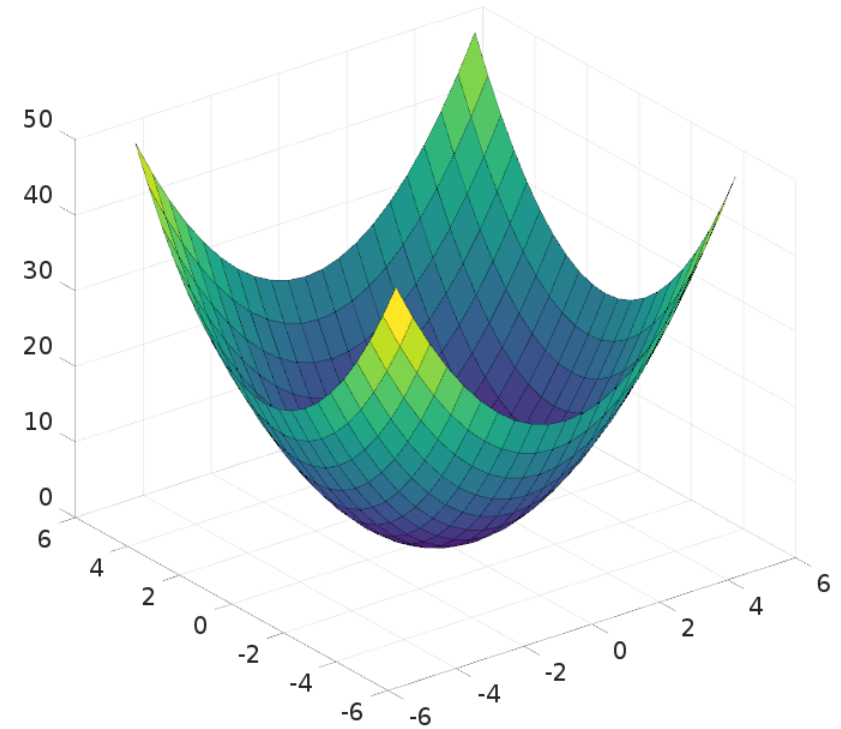
Transforming Points with a Function: Visualizing 1D

- Represented in **2D plane**:
 - Horizontal axis = input x
 - Vertical axis = output y
- Every input $\rightarrow$ single output



Transforming Points with a Function: Multiple Inputs

- Example: parabola in 3D
 $z = x^2 + y^2$
- Takes two inputs $(x, y) \rightarrow$ one output (z)
- Represented in **3D space**
- Functions map coordinates to new values
- Useful for describing **robot positions & motion**
- Builds the foundation for **coordinate transforms**



2D and 3D Functions

- Robots live in 2D (planar) or 3D (spatial) worlds
- To describe motion mathematically → need functions in higher dimensions

- Point in 2D space:

$$p_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$$

- Function: maps input → output

$$p_2 = f(p_1)$$

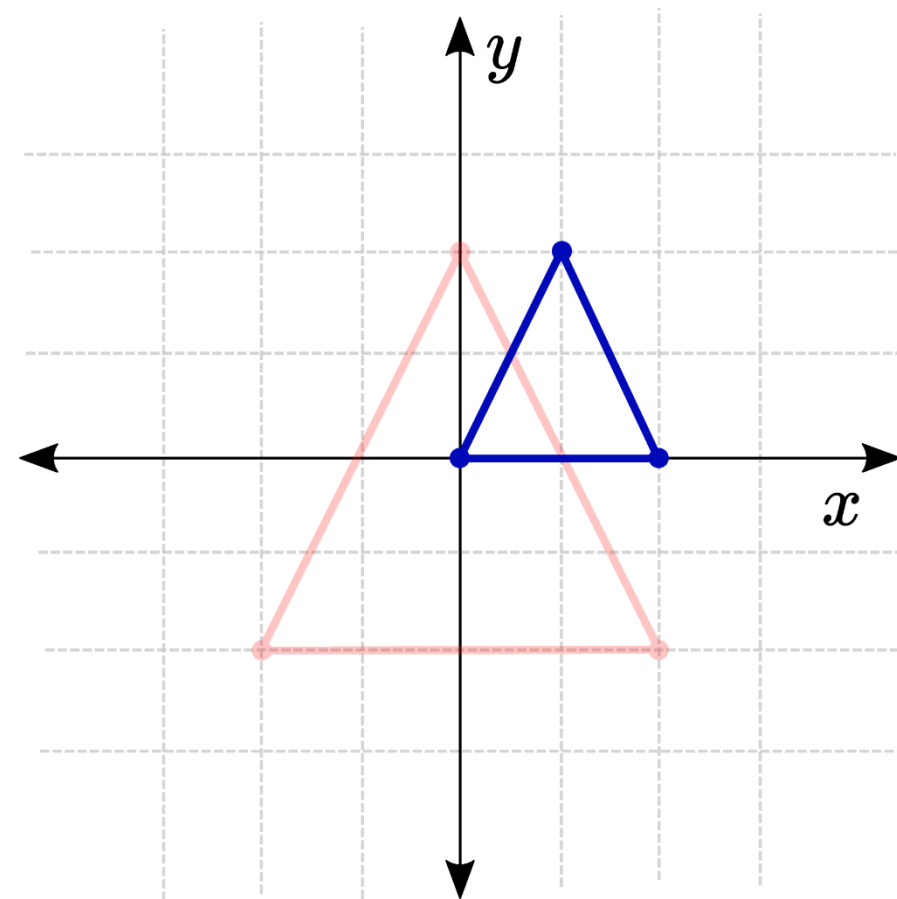
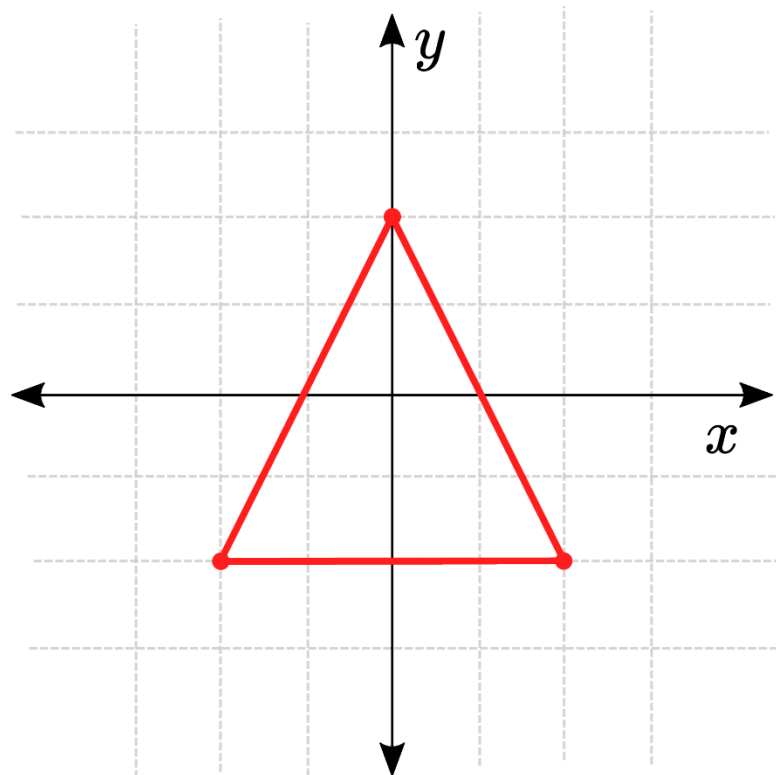
- Example:

$$f(p) = \frac{p}{2} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Input	Output
(0, 2)	(1, 2)
(2, -2)	(2, 0)
(-2, -2)	(0, 0)

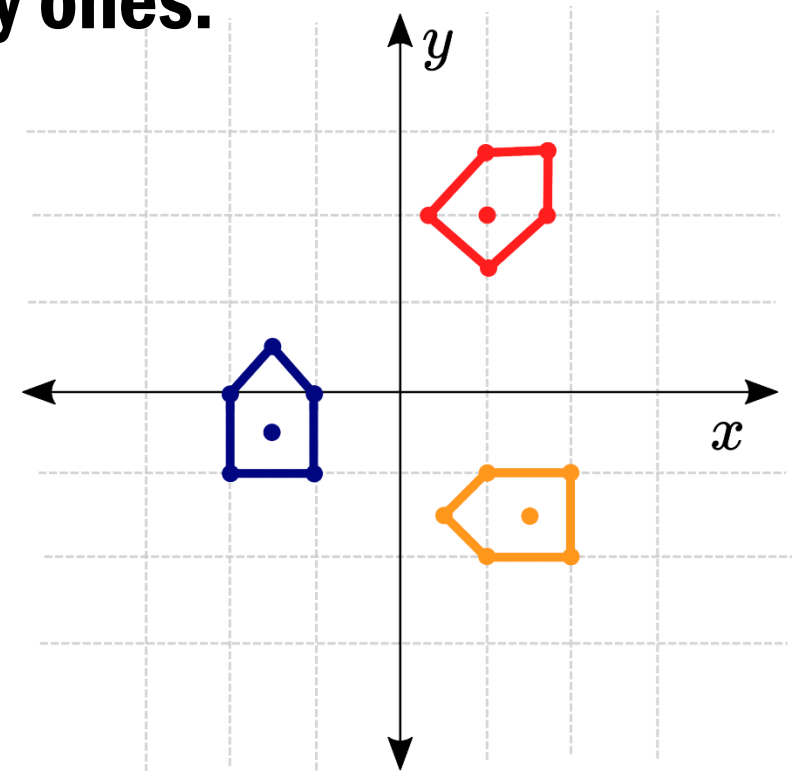
2D and 3D Functions: Example Transformation

Input	Output
$(0, 2)$	$(1, 2)$
$(2, -2)$	$(2, 0)$
$(-2, -2)$	$(0, 0)$



2D and 3D Functions

- **Infinite possible transforms, but only two key ones:**
 - **Translation (shifting position)**
 - **Rotation (changing orientation)**
- **Together → framework for robotic motion**
- **Next: Linear vs Non-Linear Transformations**
- **Tools: matrices for 2D & 3D transformations**
- **Goal: robust mathematical language for robot motion**



Linear Transformations: Introduction

- Two categories of transformations: **linear** vs **non-linear**
- Linear form:
$$f(p) = Ap$$
- A : $n \times n$ matrix
- Dimensions $\rightarrow$ 1D, 2D, 3D cases

Linear Transformations: Matrix Forms

- 1D: $[a]$
- 2D: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$
- 3D: $\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$
- Anything **not** of this form $\rightarrow$ **non-linear**

Linear Transformations: 2D Example

$$p_2 = Ap_1$$

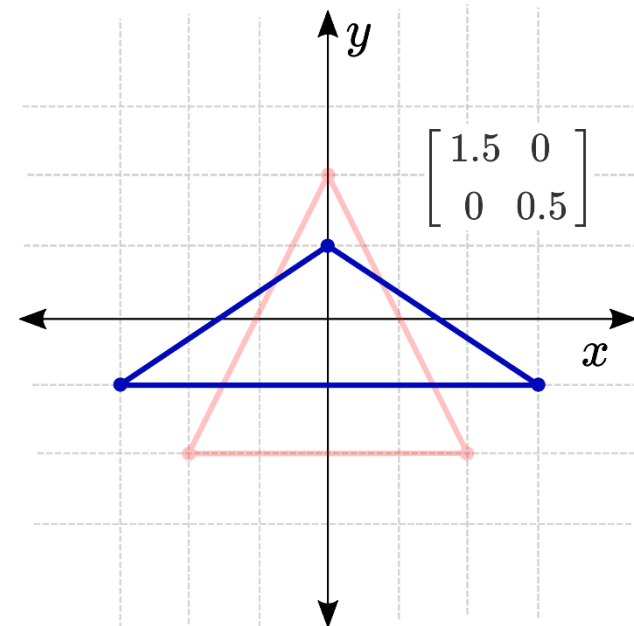
$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} ax_1 + by_1 \\ cx_1 + dy_1 \end{bmatrix}$$

Linear Transformations: Scaling

$$\begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix}$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} ax_1 \\ dy_1 \end{bmatrix}$$

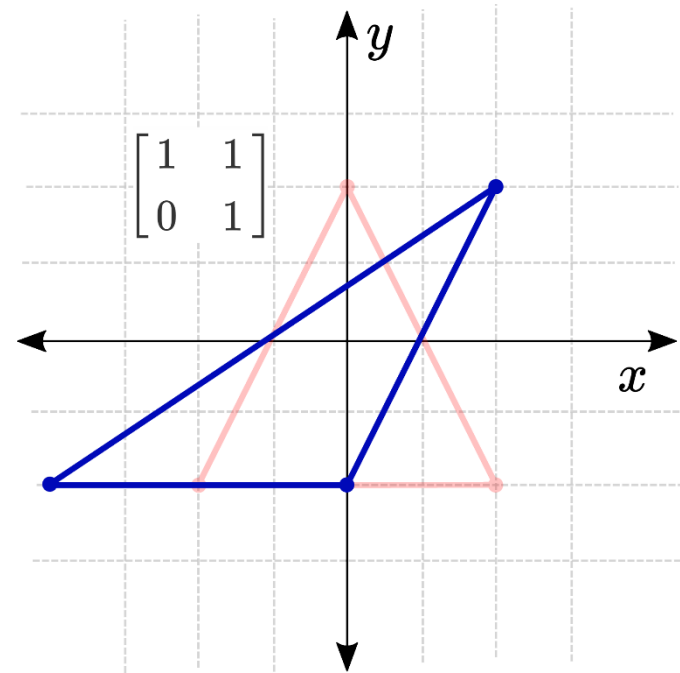
- a : scales x-axis
- d : scales y-axis
- Special case: shrink everything $\rightarrow$ origin



Linear Transformations: Shearing

$$\begin{bmatrix} 1 & b \\ c & 1 \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & b \\ c & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_1 + by_1 \\ y_1 + cx_1 \end{bmatrix}$$

- b : old y affects new x
- c : old x affects new y
- Produces **skew/shear** effect



Linear Transformations: Combinations

- Mixing elements → new effects
- Example:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Mirrors points across line $y = x$

Linear Transformations: Properties

- $0 \mapsto 0$ (origin fixed)
- Always odd: $f(-p) = -f(p) \rightarrow$ mirroring effect
- Transformations **chain by multiplication**:

$$f_3(f_2(f_1(p))) = A_3 A_2 A_1 p$$

2D Rotations: Introduction

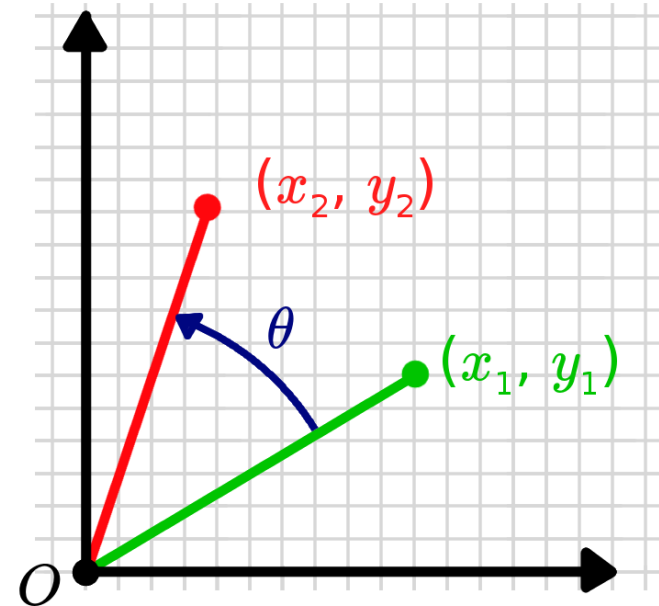
- Robots **rotate** all the time; they don't scale or mirror
- Need a clean math tool to rotate **points/point sets** in 2D
- Goal: derive the **2×2 rotation matrix** and key properties

2D Rotations: Problem Setup

- Given point (x_1, y_1)
- Rotate **anticlockwise** by angle θ about origin
- Find matrix $R(\theta)$ such that

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = R(\theta) \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

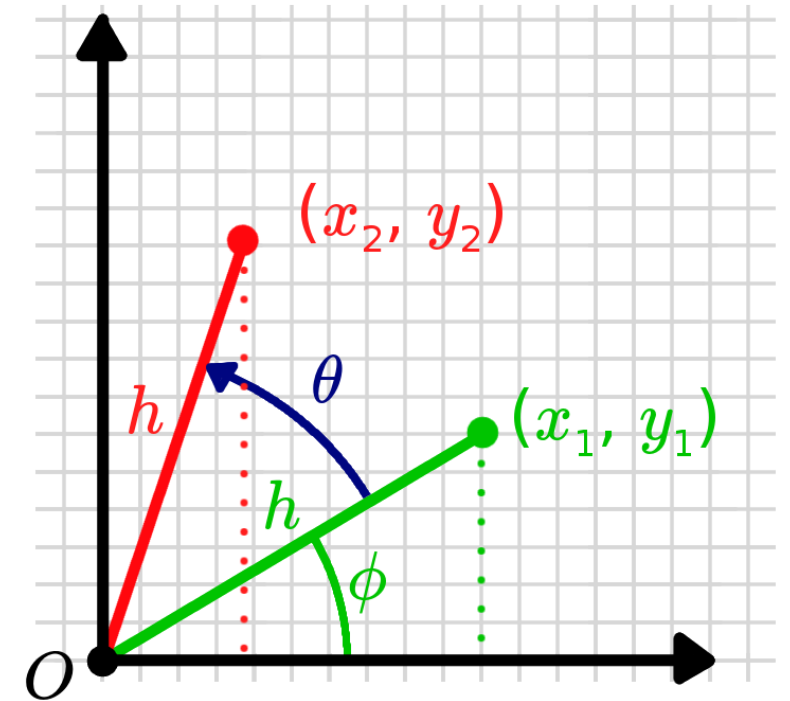
$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} ax_1 + by_1 \\ cx_1 + dy_1 \end{bmatrix}$$



2D Rotations: Polar Helper

- Let $h = \sqrt{x_1^2 + y_1^2}$, $\phi = \text{atan2}(y_1, x_1)$
- Then:

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} h \cos \phi \\ h \sin \phi \end{bmatrix}, \quad \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} h \cos(\phi + \theta) \\ h \sin(\phi + \theta) \end{bmatrix}$$



2D Rotations: Use Trig Identities

$$\cos(\phi + \theta) = \cos \phi \cos \theta - \sin \phi \sin \theta$$

$$\sin(\phi + \theta) = \sin \phi \cos \theta + \cos \phi \sin \theta$$

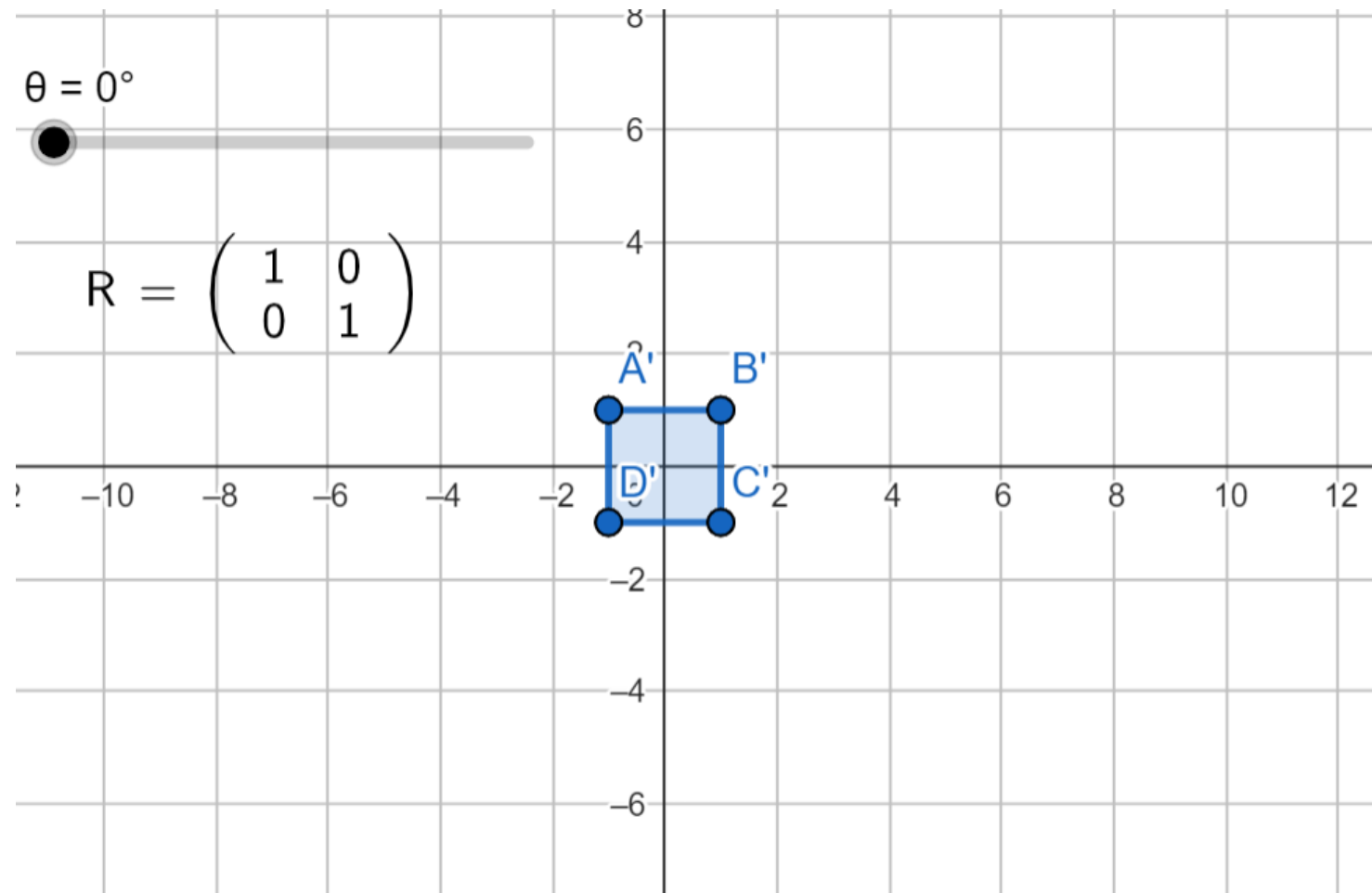
$$\Rightarrow \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} x_1 \cos \theta - y_1 \sin \theta \\ x_1 \sin \theta + y_1 \cos \theta \end{bmatrix}$$

2D Rotations: The Rotation Matrix

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

- Apply to **any point or polygon** to rotate by θ about the origin

Example: GeoGebra



2D Rotations: Property 1 – Inverse = Transpose

$$R(\theta)^{-1} = R(-\theta) = R(\theta)^{\top}$$

- Reason: $\cos(-\theta) = \cos \theta$, $\sin(-\theta) = -\sin \theta$
- Practical: undo a rotation by **transpose**

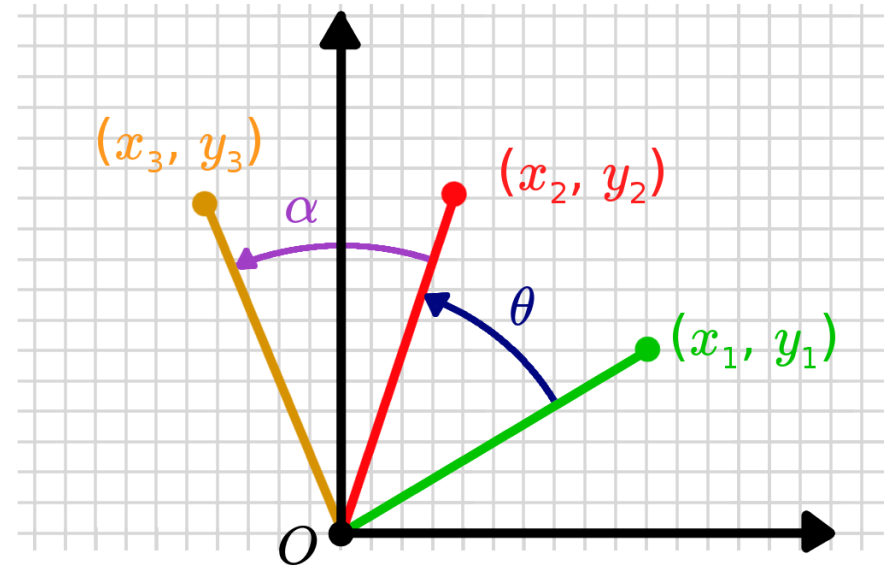
2D Rotations: Property 2 – Determinant = 1

$$\det R(\theta) = \cos^2 \theta + \sin^2 \theta = 1$$

- Area-preserving (no scaling), **orientation-preserving**

2D Rotations: Property 3 – Closed Under Multiplication

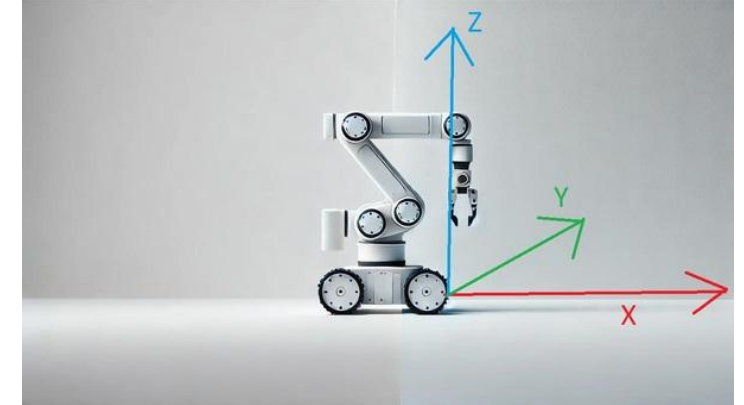
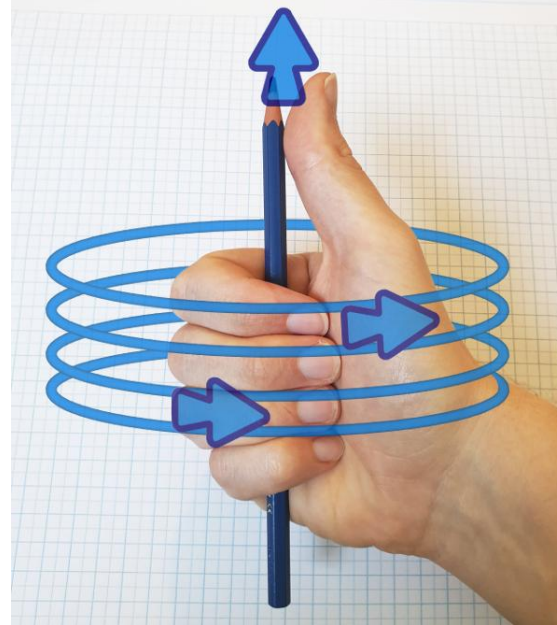
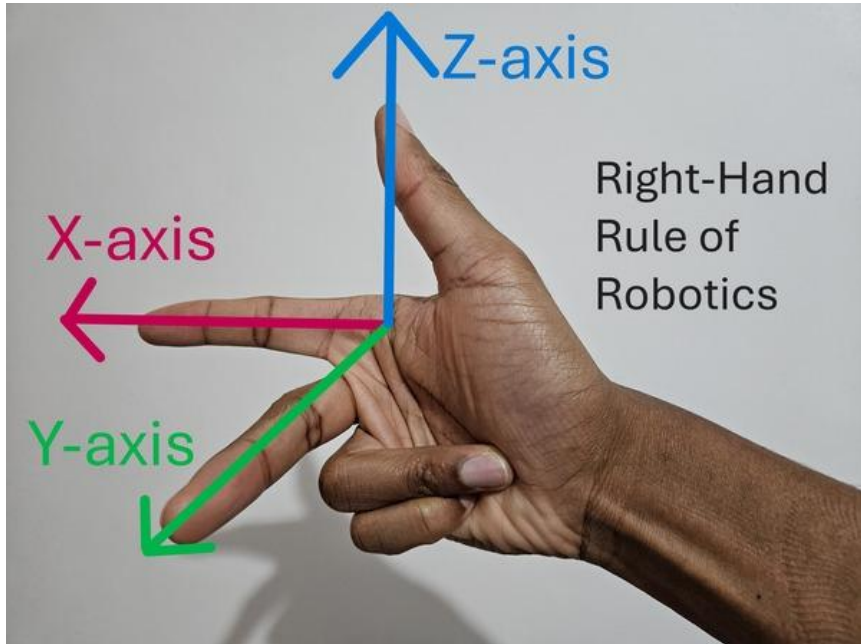
$$R(\alpha)R(\theta) = R(\alpha + \theta)$$



- Compose rotations by **adding angles**
- Clean chaining for kinematic stacks

3D Rotation: Right-Hand Rule

- Axes: X (right), Y (forward), Z (up)
- Positive rotation follows **right-hand grip rule**
- 2D rotation in $x - y$ plane $\Leftrightarrow$ rotation about Z



[Source](#)

3D Rotation:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotate About Z

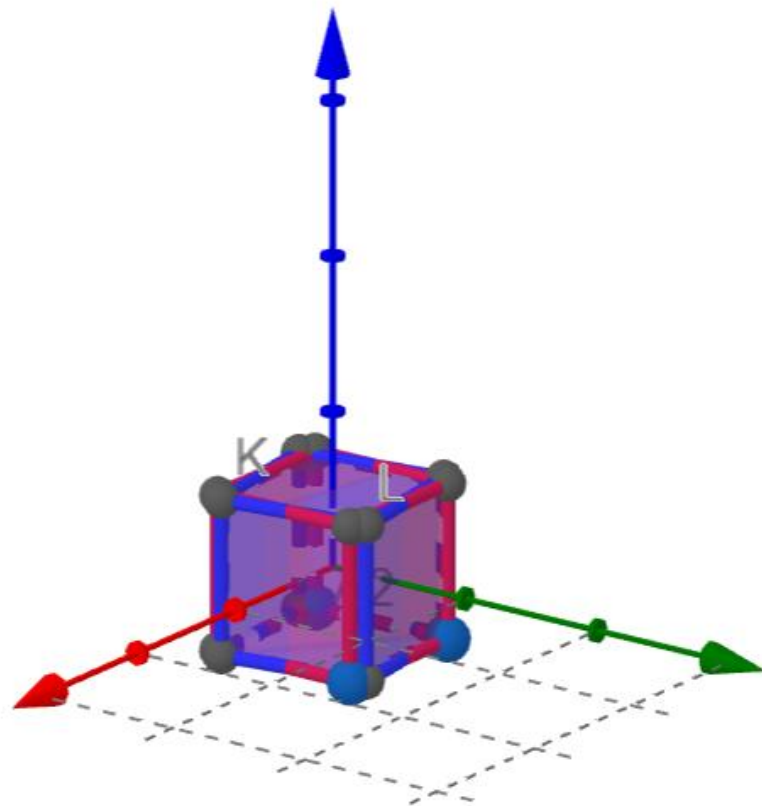
$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

Rotate About Y

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

Rotate About X

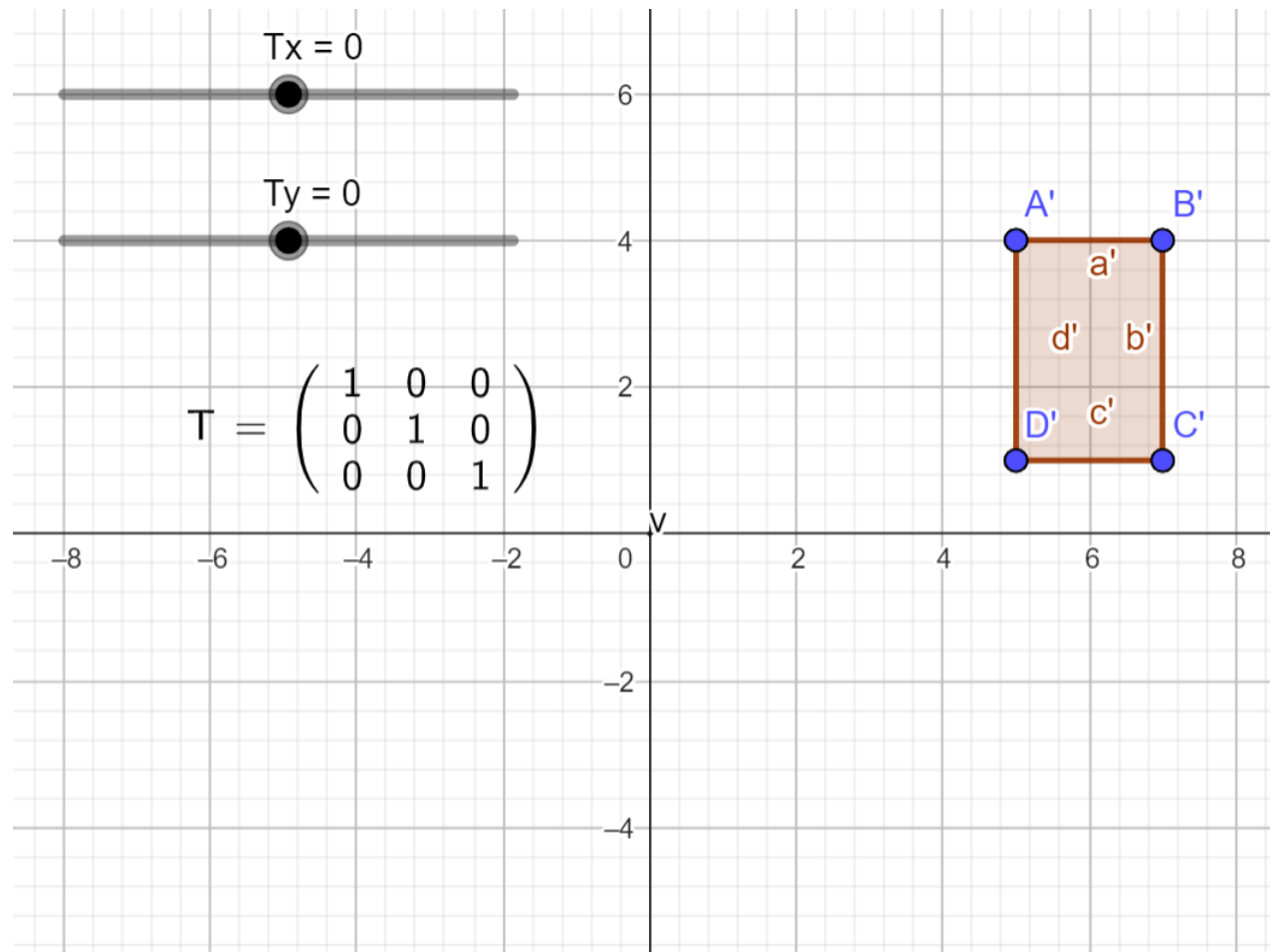
Example: GeoGebra



Translations: 2D Homogeneous Matrix

$$\bar{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \boxed{\bar{p}' = \underbrace{\begin{bmatrix} 1 & 0 & s_x \\ 0 & 1 & s_y \\ 0 & 0 & 1 \end{bmatrix}}_{T(s_x, s_y)} \bar{p}} \quad \Rightarrow \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x + s_x \\ y + s_y \\ 1 \end{bmatrix}$$

Example: GeoGebra



Translations: 3D Homogeneous Matrix

$$\bar{p}' = \underbrace{\begin{bmatrix} 1 & 0 & 0 & s_x \\ 0 & 1 & 0 & s_y \\ 0 & 0 & 1 & s_z \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{T(s_x, s_y, \downarrow)} \bar{p}, \quad \bar{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

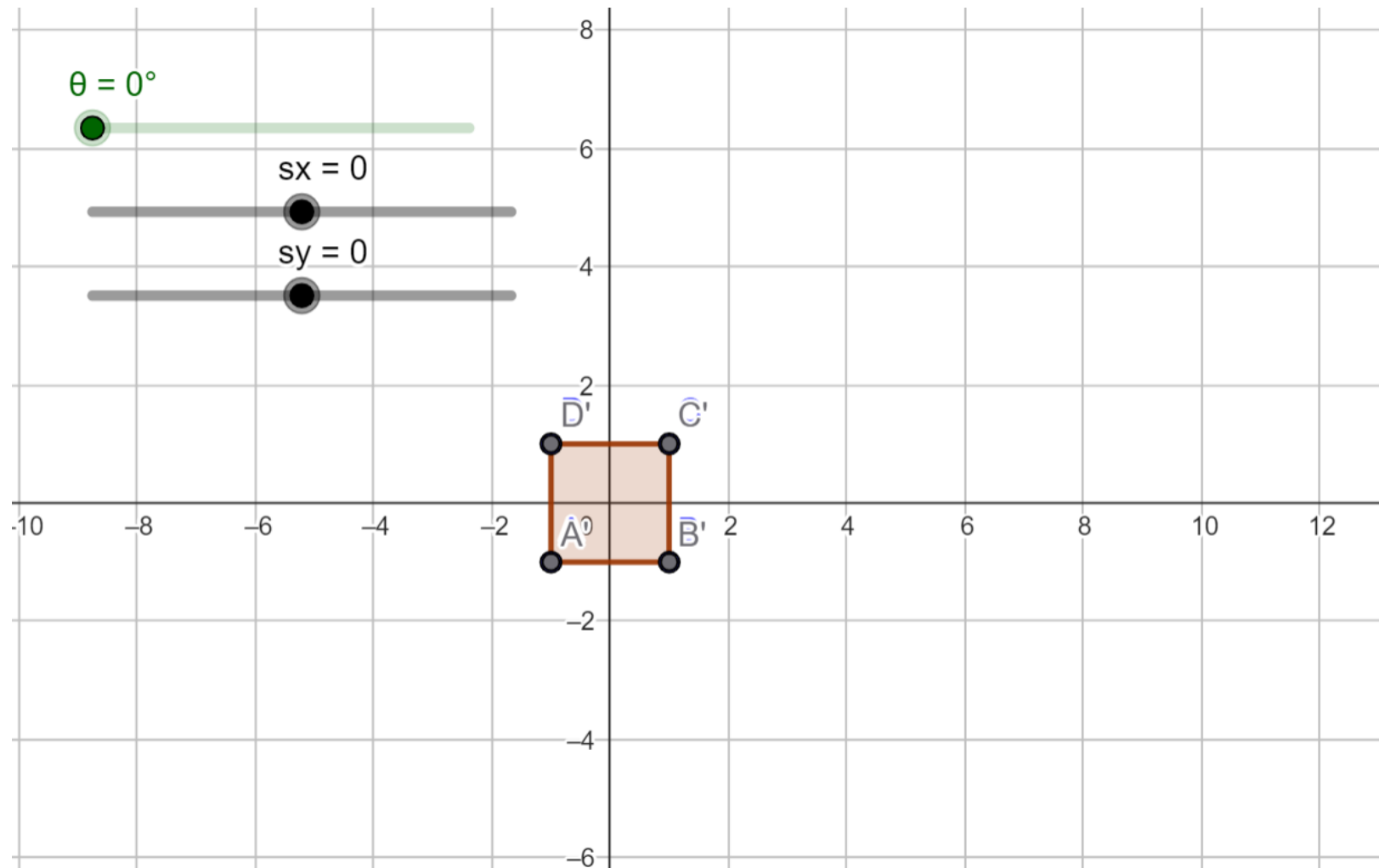
Translations: Mixing with Rotation (2D)

$$\bar{p}' = \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta & s_x \\ \sin \theta & \cos \theta & s_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Rotate then translate}} \bar{p}$$

Translations: Mixing with Rotation (3D)

$$\bar{p}' = \underbrace{\begin{bmatrix} R_{3 \times 3} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{\text{SE}(3) \text{ transform}} \bar{p}, \quad \mathbf{t} = \begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix}$$

Example: GeoGebra



Summary



- **Robots need math:** coordinate transforms = foundation for control, planning, and simulation
- **Points in space:** use vectors (2D/3D) to represent robot geometry
- **Transformations:** functions map points to new positions/orientations
- **Linear framework:** matrices capture rotation, scaling, shear; properties simplify chaining
- **2D & 3D rotations:** derived from trig, preserve distance & orientation ($SO(2)$, $SO(3)$)
- **Translations:** non-linear in raw form, solved via **homogeneous coordinates**
- **Unified representation:** rotation + translation together → single matrix multiply ($SE(2)/SE(3)$)
- Sets the stage for: **frames, kinematics, and full robot motion analysis**

THANK YOU.



WE ARE

HUMBER